

Java EE — Module 2

JDBC (Java Database Connectivity)

Detailed Study Notes

Q1. What is JDBC? Explain its need.

JDBC (Java Database Connectivity) is a standard Java API (part of Java SE) that provides a set of classes and interfaces enabling Java programs to connect to, query, and manipulate relational databases. It acts as a **bridge** between a Java application and a database.

It is defined in the package `java.sql` and `javax.sql`.

Why is JDBC Needed?

Before JDBC, Java applications had no standard way to communicate with different databases. Each database vendor had its own proprietary API, which caused problems:

- A program written for Oracle couldn't easily work with MySQL.
- Developers had to rewrite database code for every database change.
- No standard interface meant less portability and more maintenance.

JDBC solves this by:

- Providing a **uniform API** — same code works with any database that has a JDBC driver.
- Allowing **dynamic database connection** at runtime.
- Supporting **SQL execution** (DDL, DML, queries).
- Enabling **transaction management** — commit, rollback, savepoints.
- Supporting **stored procedures** via `CallableStatement`.
- Providing **metadata access** — info about the DB structure.

Real-world analogy: JDBC is like a universal remote control. No matter which brand of TV (MySQL, Oracle, PostgreSQL), the remote (JDBC) can operate it as long as it has the right adapter (driver).

Q2. Explain different types of JDBC drivers.

A **JDBC Driver** is a software component that translates Java JDBC calls into database-specific protocol calls. There are **4 types** of JDBC drivers, classified by how they communicate with the database.

Type 1 — JDBC-ODBC Bridge Driver

- Converts JDBC calls → ODBC calls → Database.

- Uses Microsoft's ODBC driver as an intermediary.
- **Pros:** Easy to use, no extra installation needed for ODBC databases.
- **Cons:** Slow (two-step translation), platform-dependent (ODBC is Windows-centric), deprecated in Java 8+.
- **Example:** `sun.jdbc.odbc.JdbcOdbcDriver`

Type 2 — Native-API / Partly Java Driver

- Converts JDBC calls → native database client library calls.
- Requires native DB client library installed on the machine.
- **Pros:** Faster than Type 1.
- **Cons:** Platform-dependent, requires native libraries, not portable.
- **Example:** Oracle OCI driver.

Type 3 — Network Protocol / Middleware Driver

- Converts JDBC calls → database-independent middleware protocol → Database.
- A middleware server handles the actual DB communication.
- **Pros:** Platform-independent, no client-side installation, supports multiple DBs.
- **Cons:** Requires middleware server, adds network overhead.
- **Example:** IDS Server driver.

Type 4 — Thin Driver / Pure Java Driver *(Most widely used)*

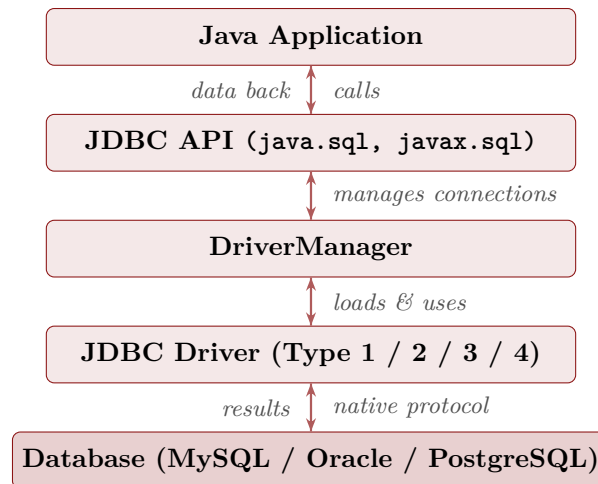
- Converts JDBC calls **directly** into the database's native protocol using pure Java.
- No native libraries, no middleware — talks directly to DB.
- **Pros:** Platform-independent, fast, easy to deploy (just a JAR file).
- **Cons:** DB-specific — need different JAR for each database.
- **Example:** MySQL Connector/J (`com.mysql.cj.jdbc.Driver`), PostgreSQL JDBC driver.

Type	Mechanism	Speed	Portability
Type 1	JDBC → ODBC → DB	Slowest	Poor
Type 2	JDBC → Native API → DB	Moderate	Poor
Type 3	JDBC → Middleware → DB	Good	Good
Type 4	JDBC → DB (direct)	Best	Best

Q3. Explain JDBC architecture with a neat diagram.

JDBC architecture defines how a Java application communicates with a database. It has

two layers: the **JDBC API layer** (used by developers) and the **JDBC Driver layer** (provided by vendors). Together they abstract the underlying database details.



Components explained:

- **Java Application** — Your program; uses JDBC API to interact with DB.
- **JDBC API** — Interfaces and classes in `java.sql`: `Connection`, `Statement`, `ResultSet`, etc.
- **DriverManager** — Central class that loads the correct driver and establishes connections.
- **JDBC Driver** — Vendor-provided implementation that translates JDBC calls to DB calls.
- **Database** — The actual relational database where data is stored.

Two-layer architecture:

Layer 1 — JDBC API: What developers interact with (high-level, standard).

Layer 2 — JDBC Driver API: How vendors plug in their driver (low-level, DB-specific).

Q4. Explain the steps to connect a Java program to a database.

Connecting a Java program to a database via JDBC involves **6 standard steps**. These steps are the same regardless of which database you use — only the driver class name and URL change.

Step 1 — Import JDBC packages

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;
```

Step 2 — Load the JDBC Driver

Registers the driver with `DriverManager`. (Not required in JDBC 4.0+ but good practice)

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 3 — Establish a Connection

Uses the `DriverManager.getConnection()` with a URL, username, and password.

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", "root", "password");
```

URL format: `jdbc:<subprotocol>://<host>:<port>/<database>`

Step 4 — Create a Statement

```
Statement stmt = con.createStatement();
```

Step 5 — Execute a Query

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
while (rs.next()) {
    System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}
```

Step 6 — Close the Connection

Always close resources in reverse order (`ResultSet` → `Statement` → `Connection`).

```
rs.close();
stmt.close();
con.close();
```

Best practice: Use `try-with-resources` (Java 7+) to auto-close connections and avoid resource leaks:

```
try (Connection con = DriverManager.getConnection(
    url, user, pass);
    Statement stmt = con.createStatement()) {
    // use stmt
} // auto-closed
```

Q5. How do you execute **SELECT**, **INSERT**, **UPDATE**, and **DELETE** queries in JDBC?

JDBC provides **three methods** on `Statement` / `PreparedStatement` for executing SQL:

- `executeQuery(sql)` — For **SELECT**; returns a `ResultSet`.
- `executeUpdate(sql)` — For **INSERT**, **UPDATE**, **DELETE**; returns `int` (rows affected).
- `execute(sql)` — For any SQL; returns `boolean` (true if `ResultSet`).

SELECT — Reading data

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    System.out.println(id + ": " + name);
}
```

INSERT — Adding a record

```
int rows = stmt.executeUpdate(
    "INSERT INTO students (id, name) VALUES (1, 'Alice')");
System.out.println(rows + " row(s) inserted.");
```

UPDATE — Modifying a record

```
int rows = stmt.executeUpdate(
    "UPDATE students SET name='Bob' WHERE id=1");
System.out.println(rows + " row(s) updated.");
```

DELETE — Removing a record

```
int rows = stmt.executeUpdate(
    "DELETE FROM students WHERE id=1");
System.out.println(rows + " row(s) deleted.");
```

Operation	Method	Returns
SELECT	executeQuery()	ResultSet
INSERT	executeUpdate()	int (rows affected)
UPDATE	executeUpdate()	int (rows affected)
DELETE	executeUpdate()	int (rows affected)

Q6. Differentiate between Statement and PreparedStatement.

Both Statement and PreparedStatement are used to execute SQL in JDBC, but they differ in how the SQL is compiled, how they handle parameters, and how secure they are.

Statement

- Used for **static SQL** — the query is written as a plain string.
- SQL is compiled **every time** it is executed.
- Cannot take parameters — values are hardcoded in the string.
- Vulnerable to **SQL injection**.

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE id=1");
```

PreparedStatement

- Used for **dynamic SQL** — uses ? as placeholders for parameters.
- SQL is compiled **once** and cached — faster for repeated execution.
- Parameters are set using `setXxx()` methods.
- **Prevents SQL injection** by treating parameters as data, not code.

```
PreparedStatement ps = con.prepareStatement(  
    "SELECT * FROM users WHERE id=?");  
ps.setInt(1, 5); // replaces ? with 5  
ResultSet rs = ps.executeQuery();
```

Feature	Statement	PreparedStatement
SQL type	Static (fixed at write-time)	Dynamic (parameters at runtime)
Compilation	Every execution	Once (pre-compiled, cached)
Parameters	Not supported	Supported via ? placeholders
Performance	Slower (repeated queries)	Faster (reuses compiled query)
SQL Injection	Vulnerable	Protected
Readability	Simple queries	Cleaner for complex queries
Use case	One-time static queries	Repeated queries, user input

Rule of thumb: Always prefer `PreparedStatement` over `Statement` whenever user input is involved.

Q7. What is ResultSet? Explain its types.

A **ResultSet** is a Java object that holds the data returned by a **SELECT** query. It acts like a **table in memory** — rows of data fetched from the database. A cursor points to the current row; you use `rs.next()` to move through the rows.

Common ResultSet methods:

- `next()` — moves cursor to next row; returns `false` at end.
- `getInt("col")`, `getString("col")`, `getDouble("col")` — retrieve column values.
- `first()`, `last()`, `previous()` — navigate (in scrollable ResultSets).

Types of ResultSet:

There are **3 types** based on scrollability and sensitivity, specified when creating a **Statement**:

1. TYPE_FORWARD_ONLY (default)

- Cursor can only move **forward** (only `next()` works).
- Cannot scroll backward or jump to a specific row.
- **Use case:** Simple one-pass data reading.

2. TYPE_SCROLL_INSENSITIVE

- Cursor can move **forward and backward**.
- Does **not reflect** live changes made to the DB after the query.
- **Use case:** When you need scrolling but don't care about real-time DB updates.

3. TYPE_SCROLL_SENSITIVE

- Cursor can move **forward and backward**.
- **Does reflect** live changes made to the DB — shows updated data.
- **Use case:** Live dashboards, real-time data.

Concurrency types (combined with above):

- CONCUR_READ_ONLY — ResultSet is read-only (default).
- CONCUR_UPDATABLE — ResultSet can be used to update the database.

```
// Scrollable, read-only ResultSet
Statement stmt = con.createStatement(
ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
rs.last();    // jump to last row
System.out.println("Total rows: " + rs.getRow());
rs.first();   // jump back to first
```

Q8. What is Metadata? Explain DatabaseMetaData and ResultSetMetaData.

Metadata means “data about data.” In JDBC, metadata refers to information **about the database or query results** — not the actual data rows, but structural information like table names, column types, driver version, etc.

1. DatabaseMetaData

Provides information about the **overall database and its structure**.

Obtained via: `DatabaseMetaData dbmd = con.getMetaData();`

Key methods:

- `getDatabaseProductName()` — returns DB name (e.g., “MySQL”).
- `getDatabaseProductVersion()` — returns DB version.
- `getDriverName()` — returns JDBC driver name.
- `getTables(catalog, schema, tableName, types)` — lists tables.
- `getUserName()` — current DB user.
- `supportsTransactions()` — whether DB supports transactions.

```
DatabaseMetaData dbmd = con.getMetaData();
System.out.println("DB: " + dbmd.getDatabaseProductName());
System.out.println("Version: " + dbmd.getDatabaseProductVersion());
System.out.println("Driver: " + dbmd.getDriverName());
```

2. ResultSetMetaData

Provides information about the **columns in a ResultSet** — useful when you don't know the query structure in advance.

Obtained via: `ResultSetMetaData rsmd = rs.getMetaData();`

Key methods:

- `getColumnCount()` — number of columns in result.
- `getColumnName(i)` — name of the *i*-th column.
- `getColumnTypeName(i)` — SQL type of column *i* (e.g., "VARCHAR").
- `getTableName(i)` — table the column belongs to.
- `isNullable(i)` — whether column allows NULL.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
ResultSetMetaData rsmd = rs.getMetaData();
int cols = rsmd.getColumnCount();
for (int i = 1; i <= cols; i++) {
    System.out.println("Column " + i + ": "
        + rsmd.getColumnName(i)
        + " (" + rsmd.getColumnTypeName(i) + ")");
}
```

Feature	DatabaseMetaData	ResultSetMetaData
Describes	The database itself	A query's result columns
Obtained from	<code>con.getMetaData()</code>	<code>rs.getMetaData()</code>
Use case	DB structure, driver info	Dynamic column inspection

Q9. Explain transaction management in JDBC.

A **transaction** is a group of SQL operations that are treated as a **single unit of work**. Either **all operations succeed** (commit) or **all are undone** (rollback). This ensures data integrity.

ACID Properties of Transactions:

- **Atomicity** — All or nothing; partial execution is not allowed.
- **Consistency** — DB moves from one valid state to another.
- **Isolation** — Concurrent transactions don't interfere with each other.
- **Durability** — Once committed, changes are permanent.

Default behavior in JDBC: Auto-commit is **true** — every statement is committed immediately.

For manual transaction control:

```
con.setAutoCommit(false); // start transaction manually

try {
    stmt.executeUpdate("UPDATE account SET bal=bal-500 WHERE id=1");
    stmt.executeUpdate("UPDATE account SET bal=bal+500 WHERE id=2");
    con.commit(); // both succeed --- commit
} catch (SQLException e) {
    con.rollback(); // one failed --- undo everything
}
```

Classic example: Bank transfer — deduct from Account A, add to Account B. If the second operation fails, the first must also be undone. Without transaction management, Account A loses money with no corresponding credit.

Q10. What is commit, rollback, and savepoint?

These three operations are the core controls of JDBC transaction management:

1. commit()

- Makes all changes in the current transaction **permanent** in the database.
- Called after all operations succeed.
- Method: `con.commit()`;

2. rollback()

- **Undoes** all changes made since the last commit.
- Called in a `catch` block when an error occurs.
- Method: `con.rollback()`;

3. Savepoint

- A **savepoint** is a **named checkpoint** within a transaction.
- You can rollback to a savepoint instead of rolling back the entire transaction.
- Useful for long transactions where only partial rollback is needed.

```
con.setAutoCommit(false);
stmt.executeUpdate("INSERT INTO orders VALUES (1, 'Laptop')");

Savepoint sp1 = con.setSavepoint("afterLaptop"); // mark checkpoint

stmt.executeUpdate("INSERT INTO orders VALUES (2, 'Phone')");

// Oops --- something wrong with Phone insert
con.rollback(sp1); // undo Phone insert, keep Laptop insert
```

```
con.commit(); // commit Laptop insert permanently

con.releaseSavepoint(sp1); // optional: release savepoint
```

Method	Effect	Analogy
<code>commit()</code>	Permanently saves all changes	Press "Save"
<code>rollback()</code>	Undoes all uncommitted changes	Press "Undo All"
<code>rollback(sp)</code>	Undoes to a specific savepoint	"Undo to checkpoint"
<code>setSavepoint()</code>	Creates a named checkpoint	Bookmark a page

Q11. Explain batch processing in JDBC with an example.

Batch processing in JDBC means grouping multiple SQL statements and sending them to the database **in one go** instead of executing each one individually. This dramatically reduces the number of database round-trips, improving performance.

Why use batch processing?

- Inserting 1000 rows one by one = 1000 network round-trips to DB.
- Batch insert of 1000 rows = 1 or a few round-trips.
- Result: **much faster** for bulk operations.

Key methods:

- `addBatch(sql)` — adds statement to the batch queue.
- `executeBatch()` — sends and executes the entire batch; returns `int[]` of row counts.
- `clearBatch()` — clears all statements from the batch.

Batch with Statement:

```
con.setAutoCommit(false);
Statement stmt = con.createStatement();

stmt.addBatch("INSERT INTO students VALUES (1, 'Alice', 90)");
stmt.addBatch("INSERT INTO students VALUES (2, 'Bob', 85)");
stmt.addBatch("INSERT INTO students VALUES (3, 'Carol', 92)");

int[] result = stmt.executeBatch(); // sends all at once
con.commit();
System.out.println("Rows inserted: " + result.length);
```

Batch with PreparedStatement (preferred):

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO students (id, name, marks) VALUES (?, ?, ?)");

for (int i = 1; i <= 100; i++) {
    ps.setInt(1, i);
```

```

        ps.setString(2, "Student" + i);
        ps.setInt(3, 70 + i % 30);
        ps.addBatch(); // add this set of params to batch
    }

    int[] counts = ps.executeBatch(); // execute all 100 at once
    con.commit();

```

Best practice: Always disable auto-commit before batch processing and commit after `executeBatch()` for all-or-nothing atomicity. Also, use `PreparedStatement` batches over `Statement` batches — they are safer and faster.

Q12. Discuss SQL injection and how `PreparedStatement` prevents it.

SQL Injection is a security attack where a malicious user inserts harmful SQL code into an input field, tricking the application into executing unintended SQL commands. It is one of the most common and dangerous web vulnerabilities.

How SQL injection works — Example:

Suppose a login query is built like this:

```

String query = "SELECT * FROM users WHERE username='"
+ username + "' AND password='" + password + "'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(query);

```

If the attacker types:

- `username = admin' -`
- `password = anything`

The query becomes:

```

SELECT * FROM users WHERE username='admin' --' AND password='...'

```

The `-` comments out the password check! The attacker logs in as admin without knowing the password.

Consequences of SQL Injection:

- Unauthorized access to user accounts.
- Data theft — dump entire tables using `UNION` attacks.
- Data destruction — `DROP TABLE students` via injected input.
- Authentication bypass.

How `PreparedStatement` prevents SQL injection:

```
PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM users WHERE username=? AND password=?");
ps.setString(1, username); // even if username = "admin' --"
ps.setString(2, password); // treated as a string literal, not code
ResultSet rs = ps.executeQuery();
```

Why it's safe:

1. The SQL structure is **pre-compiled** before parameters are set.
2. Parameters are passed **separately** as data, not as part of SQL code.
3. Special characters like ' , - , ; are **automatically escaped**.
4. The DB sees: `username = "admin' --"` as a literal string, not SQL syntax.

Aspect	Statement (vulnerable)	PreparedStatement (safe)
Input handling	Concatenates directly into SQL	Treats input as pure data
Injection risk	High	None (parameterized)
Special chars	Interpreted as SQL	Automatically escaped
Performance	Recompiles every time	Pre-compiled, cached

Key principle: `PreparedStatement` separates *code* from *data*. No matter what a user types into a field, it can **never become SQL code** when using parameterized queries. This is the single most important JDBC security practice.